

Java Lab Record: File Handling - Pattern Frequency Count

Objective / Aim

To write a Java program that **writes** the given text into a file called `sample.txt`, **appends** a final line to it, **reads** the file back, and finds the **frequency count** of the patterns `pe` and `pi`, using the file I/O classes of Java.

Problem Statement

Write the below text in the file called `sample.txt` and then find the frequency count of the patterns `pe` and `pi`.

```
Peter Piper picked a peck of pickled peppers
A peck of pickled peppers Peter Piper picked
If Peter Piper picked a peck of pickled peppers
Where's the peck of pickled peppers Peter Piper picked?
```

Expected Output

```
'pe' - no of occurrences - 20
'pi' - no of occurrences - 12
```

System Requirements

Hardware

```
Standard PC / Laptop
Intel Core i3 / i5 processor or equivalent
Minimum 4 GB RAM
Minimum 500 MB free storage
```

Software

```
Operating System : Windows / Linux / macOS
Java Development Kit : JDK 17 or JDK 21
Text Editor or IDE : Eclipse, IntelliJ IDEA, NetBeans, or VS Code
Command Prompt / Terminal
```

☐ Algorithm / Logic Description

1. Create a class named FileHandling in package beyond.
2. Create a static method countOccurrences(text, pattern):
 - a. Convert both the text and the pattern to lower case so that 'Pe' and 'pe' are treated as the same pattern.
 - b. Set count = 0 and index = 0.
 - c. Repeat: find the pattern starting from 'index'.
 - If found, increase count and move index past the whole match (index + pattern length).
 - If not found (-1), stop.
 - d. Return count.
3. In main(), create the path of the file: src/beyond/sample.txt.
4. WRITE MODE: write the first three lines into the file, replacing any old content (CREATE + TRUNCATE_EXISTING).
5. APPEND MODE: add the final line at the end of the file (APPEND).
6. READ MODE: read the complete file back into one String.
7. Call countOccurrences() for "pe" and for "pi".
8. Display both counts.

Real-time Application

Log file analysis, word and keyword frequency counting, search features in text editors, and plagiarism / pattern detection tools.

FileHandling.java

Source Code

```
package beyond;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.util.Locale;

public class FileHandling {

    public static int countOccurrences(String text, String pattern) {
        String lowerText = text.toLowerCase(Locale.ROOT);
        String lowerPattern = pattern.toLowerCase(Locale.ROOT);
        int count = 0;
        int index = 0;

        // Move past each full match so every occurrence is counted once.
        while ((index = lowerText.indexOf(lowerPattern, index)) != -1) {
            count++;
            index += lowerPattern.length();
        }
    }
}
```

```

        return count;
    }

    public static void main(String[] args) throws IOException {
        Path file = Path.of("src", "beyond", "sample.txt");
        String firstLines = String.join(System.lineSeparator(),
            "Peter Piper picked a peck of pickled peppers",
            "A peck of pickled peppers Peter Piper picked",
            "If Peter Piper picked a peck of pickled peppers");
        String finalLine = "Where's the peck of pickled peppers Peter Piper
picked?";

        // Write mode replaces any old contents so repeated runs stay
        consistent.
        Files.writeString(file, firstLines, StandardCharsets.UTF_8,
            StandardOpenOption.CREATE,
            StandardOpenOption.TRUNCATE_EXISTING);
        // Append the final line to demonstrate append mode without changing
        the sample text.
        Files.writeString(file, System.lineSeparator() + finalLine,
            StandardCharsets.UTF_8,
            StandardOpenOption.APPEND);

        // Read mode loads the completed file for pattern counting.
        String fileContents = Files.readString(file, StandardCharsets.UTF_8);

        System.out.println("'pe' - no of occurrences - "
            + countOccurrences(fileContents, "pe"));
        System.out.println("'pi' - no of occurrences - "
            + countOccurrences(fileContents, "pi"));
    }
}

```

1. Concept and Exception in Simple Words

Concepts used

Files.writeString()	-> writes a String into a file in one call.
Files.readString()	-> reads a whole file back as one String.
Path.of(...)	-> builds the file location (src/beyond/sample.txt).
StandardOpenOption	-> tells Java HOW to open the file:
CREATE	create the file if it does not exist
TRUNCATE_EXISTING	empty the file first (this is "write mode")
APPEND	add at the end, keep old content ("append mode")
StandardCharsets.UTF_8	-> the character encoding. It is needed because the last line contains a curly apostrophe ('), which is not a plain ASCII character.
Locale.ROOT	-> makes toLowerCase() behave the same on every computer, whatever its language settings.
static method	-> countOccurrences() belongs to the class, so main() calls it without creating an object.

Why the text is written in two steps

The first three lines are written in **write mode** (the file is emptied first), and the fourth line is then added in **append mode**. This shows both modes. Writing in write mode first also makes every run give the same result, because old content from a previous run is erased.

Why the count is not case-sensitive

The program converts the text to lower case before counting. So `Peter` counts for `pe` and `Piper` counts for `pi`. A case-sensitive search would give smaller numbers and would not match the expected 20 and 12.

Exception in simple words

File reading and writing can fail for reasons outside your program (missing folder, no permission, bad data). Java signals this with `IOException`, a **checked exception**, which means the compiler forces you to deal with it. This program deals with it by declaring `throws IOException` on `main()` instead of using try-catch. If something goes wrong, the program stops and prints the error.

Exception	Subclass of	When it could happen here
<code>NoSuchFileException</code>	<code>IOException</code>	The folder <code>src/beyond</code> does not exist (Java creates the file, not the missing folders)
<code>AccessDeniedException</code>	<code>IOException</code>	The folder or file is read-only
<code>MalformedInputException</code>	<code>IOException</code>	<code>readString()</code> finds bytes that are not valid UTF-8
<code>NullPointerException</code>	<code>RuntimeException</code>	Only if <code>countOccurrences()</code> were called with <code>null</code>

Important: the path is relative, so the program must be run from the **project root folder** (the folder that contains `src`). If `src/beyond` is missing, create it first, or you will get `NoSuchFileException`.

2. Step-wise Execution

1. Program starts in `main()`.
2. `file = Path.of("src", "beyond", "sample.txt")`
3. `firstLines` = the first three sentences joined with a line separator (no separator after the third line).
4. `finalLine = "Where's the peck of pickled peppers Peter Piper picked?"`
5. WRITE MODE:
 `Files.writeString(file, firstLines, UTF_8, CREATE, TRUNCATE_EXISTING)`
 -> `sample.txt` now holds exactly 3 lines.
6. APPEND MODE:
 `Files.writeString(file, separator + finalLine, UTF_8, APPEND)`
 -> the 4th line is added at the end. The file now has 4 lines.
7. READ MODE:
 `fileContents = Files.readString(file, UTF_8)`
 -> the whole file is loaded into one `String`.

8. `countOccurrences(fileContents, "pe")` is called:
 - a. text and pattern are converted to lower case.
 - b. `count = 0, index = 0`
 - c. `indexOf("pe", index)` finds a match -> `count++`,
index moves forward by 2 (length of "pe").
 - d. This repeats until `indexOf` returns -1 (no more matches).
 - e. Returns 20.
9. `countOccurrences(fileContents, "pi")` works the same way -> returns 12.
10. Both results are printed.

Counting trace for line 1: "Peter Piper picked a peck of pickled peppers"

Word	'pe' matches	'pi' matches
Peter	1	0
Piper	1 (pi-PE-r)	1 (PI-per)
picked	0	1
peck	1	0
pickled	0	1
peppers	2 (PE-p-PE-rs)	0

Line total	5	3

Count for every line (case-insensitive)

Line 1	Peter Piper picked a peck of pickled peppers	pe=5	pi=3
Line 2	A peck of pickled peppers Peter Piper picked	pe=5	pi=3
Line 3	If Peter Piper picked a peck of pickled peppers	pe=5	pi=3
Line 4	Where's the peck of pickled peppers Peter Piper picked?	pe=5	pi=3

TOTAL		pe=20	
pi=12			

3. Algorithm

START

1. Create Path file = `src/beyond/sample.txt`
2. Create firstLines (3 sentences joined by line separator)
3. Create finalLine (4th sentence)
4. WRITE firstLines to file (CREATE, TRUNCATE_EXISTING, UTF-8)
5. APPEND line separator + finalLine to file (APPEND, UTF-8)
6. READ the whole file into fileContents (UTF-8)
7. CALL `countOccurrences(fileContents, "pe")` and PRINT the result
8. CALL `countOccurrences(fileContents, "pi")` and PRINT the result

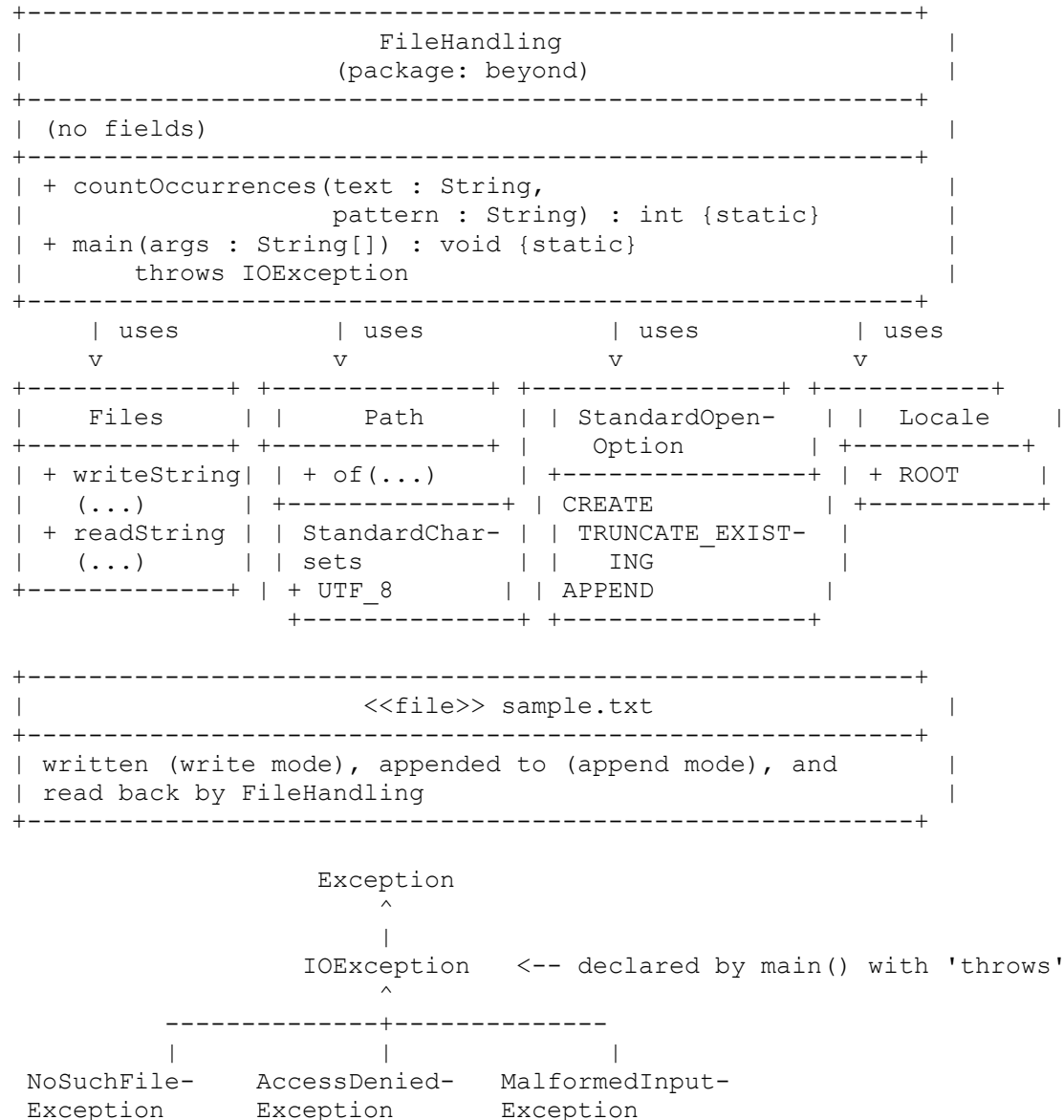
STOP

`countOccurrences(text, pattern):`

1. `lowerText = text.toLowerCase()`
2. `lowerPattern = pattern.toLowerCase()`
3. SET `count = 0, index = 0`
4. WHILE (`index = lowerText.indexOf(lowerPattern, index) != -1`):
 `count = count + 1`
 `index = index + lowerPattern.length()`
5. RETURN `count`

Time complexity : $O(n * m)$ (n = text length, m = pattern length)
 Space complexity : $O(n)$ (the file is held in memory as one String)

4. UML Class Diagram



Legend: + public, {static} belongs to the class itself.

Compilation and Execution

Folder structure (run commands from the project root):

```
project/
|-- src/
    |-- beyond/
        |-- FileHandling.java
        |-- sample.txt      (created by the program)
```

Compile:

```
javac -d out src/beyond/FileHandling.java
```

Run (from the project root, so the path src/beyond/sample.txt is found):

```
java -cp out beyond.FileHandling
```

Sample Output

```
'pe' - no of occurrences - 20
'pi' - no of occurrences - 12
```

Output:

```
PS C:\Users\Admin\OneDrive\Documents\basicjava> c:\cd "C:\Users\Admin\OneDrive\Documents\basicjava" & "C:\Program Files\Java\jdk-26.0.1\bin\java.exe" --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -cp "C:\Users\Admin\OneDrive\Documents\basicjava\bin" beyond.FileHandling
'pe' - no of occurrences - 20
'pi' - no of occurrences - 12
```

Contents of sample.txt after the run

```
Peter Piper picked a peck of pickled peppers
A peck of pickled peppers Peter Piper picked
If Peter Piper picked a peck of pickled peppers
Where's the peck of pickled peppers Peter Piper picked?
```

Result

The File Handling program was successfully implemented in Java using the file I/O classes `Files`, `Path`, and `StandardOpenOption`. The program wrote the given text into `sample.txt` in write mode, appended the final line in append mode, read the file back, and counted the patterns using a static method. It correctly displayed 20 occurrences of 'pe' and 12 occurrences of 'pi', matching the expected output.